

Home Library - Take-Home Assignment

Welcome! **Fork the repository** at <https://github.com/marianchelmus/home-library-wa> and build your solution in your fork. This starter gives you the infrastructure (PostgreSQL + RabbitMQ via Docker Compose) and sample data, so you can spend your time on the app rather than the plumbing.

Target effort: ~4 hours. We value a small, clean, working solution over a large unfinished one. If something is unclear, make a reasonable assumption, note it, and keep moving.

What's in this starter

File	Purpose
<code>docker-compose.yml</code>	PostgreSQL + RabbitMQ, ready to run, with commented service templates for your API, worker, and web app to fill in.
<code>.env.example</code>	Environment variables used by the compose file (copy to <code>.env</code>).
<code>db/init.sql</code>	An optional reference schema for the <code>library</code> table - use it, or use EF Core migrations.
<code>sample-books-10.csv</code> , <code>sample-books-20.csv</code>	Sample data to import (<code>name</code> , <code>author</code> , <code>genre</code>).

Quick start

```
# 1. copy the env defaults
cp .env.example .env

# 2. bring up the infrastructure
docker compose up -d
```

- RabbitMQ management UI → `http://localhost:15672` (`guest` / `guest`)
- PostgreSQL → `localhost:5432` (`library` / `library`, database `library`)

Build your API, worker, and Angular app. Once they're containerised, **edit the** `api` / `worker` / `web` **templates** in `docker-compose.yml` so that a single `docker compose up -d --build` brings up the whole application.

Overview

Build a small **bulk-import** web application for a book library. A user **drag-and-drops a CSV** of books into the Angular UI; the **.NET API** publishes the rows to **RabbitMQ**; a **separate worker service** consumes them and inserts each book into a **PostgreSQL** `library` table. The UI also shows a **list of**

imported books - name, author, genre, and import date. Everything is wired together with **Docker Compose** so the whole app comes up with a single `docker compose up -d --build`.

This mirrors how we actually work: an API that hands work off to a queue, a separate worker that does the persistence, and an Angular portal on top. We care far more about clean, correct, well-reasoned code than about feature count - **polish beyond the requirements is not expected**.

Requirements

Server (.NET / ASP.NET Core)

1. Build a REST API with **.NET 10 and ASP.NET Core** (C#).
2. Implement these endpoints: - `POST /api/imports` - accepts an uploaded CSV (`multipart/form-data`), parses the rows, and **publishes them to RabbitMQ**. The API must **not** insert books into the database itself. Return promptly (e.g. `202 Accepted`) with the number of rows queued. - `GET /api/books` - returns the books in the `library` table (name, author, genre, import date), ordered by import date descending.
3. The CSV has three columns - **name, author, genre** - with a header row (see the provided `sample-books-10.csv` and `sample-books-20.csv`).
4. Handle the obvious errors sensibly - no file, wrong content type, empty CSV - with appropriate status codes.

Database (PostgreSQL)

1. A single table, `library` , with at least: an id, `name` , `author` , `genre` , and `import_date` .
2. `import_date` is the moment the book was imported (set when the row is inserted), stored in UTC.
3. Use whichever data-access approach you prefer - **EF Core** (matches our stack), Dapper, or Npgsql. How the schema is created is your call: an EF migration or the provided `db/init.sql` init script are both fine.

Messaging (RabbitMQ)

1. The API **publishes** the parsed rows to a RabbitMQ queue - e.g. one `BookImport` message per row carrying `{ name, author, genre }` . Keep messages small (the rows, not the raw file).
2. A **separate consumer/worker service** consumes those messages and **inserts each book** into the `library` table, stamping `import_date` . The worker must be a **genuinely separate runnable process/project** - not a `BackgroundService` hosted inside the API - and it must go through a real RabbitMQ broker (no in-process/in-memory queue).
3. This producer → queue → consumer handoff is the main thing we're assessing: **the API parses and publishes; the worker persists**.

Client (Angular)

1. Build the UI with **Angular** (via `@angular/cli`). This is required - it's our stack.

2. Implement these features: - A **drag-and-drop** area to drop a CSV file (a click-to-browse fallback is welcome), which uploads to `POST /api/imports`. - A **list/table of books** showing **name, author, genre, and import date**, populated from `GET /api/books`. - The list should reflect newly imported books. Because the worker inserts rows asynchronously, **poll** `GET /api/books` **every couple of seconds** after an upload rather than refreshing once.
3. **Styling** - use Tailwind CSS
4. **Dependencies** - besides your styling choice and whatever `@angular/cli` scaffolds, avoid extra 3rd-party packages. Plain Angular **signals** are a great fit here; NgRx is unnecessary.

Docker

1. Extend the provided `docker-compose.yml` so it runs the **entire application** - PostgreSQL, RabbitMQ, the API, the worker, and the Angular app - so that a single `docker compose up -d --build` brings everything up and the app is usable in the browser with no further steps.
2. Include a `Dockerfile` for the API, the worker, and the frontend.
3. For local development you may also run the services directly (`dotnet run` , `ng serve`), but the path we grade is the single-command `docker compose up -d --build`.

Assumptions You Can Make

To keep you moving - make these calls without overthinking them (and feel free to note your own):

- The CSV always has a header row; skip it.
- The `id` type is your choice (e.g. integer identity or UUID).
- Skip malformed or incomplete rows; the valid rows should still import. No de-duplication is required.
- `GET /api/books` is **eventually consistent** - books appear a moment after upload, once the worker has processed them. That short delay is expected and fine.
- "The API must not write to the DB" refers to **book inserts** (those happen only in the worker); reading for `GET /api/books` from the API is fine.
- Assume small CSVs (tens to low-hundreds of rows) - no batching, backpressure, or streaming needed.

Bonus Points (only if you have time)

1. Drag-and-drop polish - highlight on drag-over, reject non-CSV files.
2. A few unit tests.
3. Graceful handling of RabbitMQ / PostgreSQL not being ready yet (connection retry on startup).

What We're Looking For

- A clean **producer** → **RabbitMQ** → **consumer** split, with the **worker (not the API) doing the database writes**.
- **Clear, readable code** with sensible types (records/DTOs).

- A **working, easy-to-run app** over a feature-packed broken one.

Definition of Done

A quick self-check before you submit:

1. Dropping a CSV in the UI results in the books appearing in the list (within a few seconds), each with name, author, genre, and import date.
2. The API process never inserts into PostgreSQL - the **separate worker** does, after consuming from RabbitMQ.
3. A single `docker compose up -d --build` brings up the entire app (PostgreSQL, RabbitMQ, API, worker, and UI), usable in the browser with no further steps.

Submission

1. **Fork the repository** at <https://github.com/marianchelmus/home-library-wa> and build your solution in your fork.
2. Add a short `SOLUTION.md` (or a section at the top of this README) covering: how to run it, any assumptions or design decisions you made, and anything you'd improve with more time.
3. Make sure `docker compose up -d --build` brings up the whole stack.
4. Send us the link to your fork. If your fork is private, grant read access to `marianchelmus`.

Evaluation Criteria

1. Code quality and clarity.
2. Functionality and completeness for the given scope.
3. The RabbitMQ producer/consumer design - the worker does the persistence.
4. Usability and overall polish.
5. Proper use of C# / .NET, Angular, PostgreSQL, and 3rd-party packages.

Good luck - we're looking forward to seeing how you think.